ProcureSync — Project Status Report

Date: 9 September 2026 | **Overall Progress: 72 / 100**

1. Architecture Overview

Layer	Tech	Hosted On	URL
Frontend	React + Vite + TS	Render (Static Site)	expense-frontend-rxv8.onrender.com
Backend	FastAPI + SQLAlchemy	Render (Web Service)	expense-backend-1mpl.onrender.com
Database	SQLite (temporary)	Embedded in Render container	—
Target DB	PostgreSQL	Supabase (qagextokkrxrrmowyser)	Not connected yet
Auth	Google OAuth + Vendor OTP/MFA	—	—

2. What's Working ✅

2.1 Deployment

- Backend is **live** — `/health` returns `{"status":"healthy"}`
- Frontend is **live** — serves React SPA correctly
- Frontend `VITE_API_URL` points to correct backend
- Auto-deploy from `main` branch active
- SPA routing fixed — all frontend routes return 200

2.2 Authentication (Demo Mode)

All 6 Quick Access Profiles work:

Profile	Email	Status
Alex Rivera (ADMIN)	y.raja@dataeko.ai	✅
Sarah Chen (CEO)	ceo@dataeko.ai	✅
Marcus Johnson (CFO)	cfo@dataeko.ai	✅
Elena Rodriguez (FINANCE)	finance@dataeko.ai	✅
David Kim (MANAGER)	manager@dataeko.ai	✅
Rachel Foster (EMPLOYEE)	employee@dataeko.ai	✅

- JWT + refresh tokens issued correctly

- Custom @dataeko.ai email input works
- Vendor OTP/MFA flow implemented

2.3 API Endpoints (Tested on Production)

Endpoint	Status		Endpoint	Status
GET /health	✅		GET /vendors	✅
POST /auth/google	✅		GET /rfq	✅
GET /auth/me	✅		GET /purchase-orders	✅
POST /auth/refresh	✅		GET /grns	✅
GET /dashboard	✅		GET /contracts	✅
GET /expenses	✅		GET /deliveries	✅
GET /pr	✅		GET /notifications	✅
GET /users	✅		GET /reports	✅
GET /roles	✅		GET /analytics/spend	✅
GET /departments	✅		GET /search	✅
GET /invoices	✅		GET /system/health	✅
GET /settings	✅		GET /assets	✅

2.4 Frontend Pages — 25 pages built

Dashboard, Expenses, Purchase Requisitions, Approvals, Assets, My Assets, Vendors, Vendor Portal, Invoices, Purchase Orders, Quotations, RFQs, Contracts, Deliveries, Delivered Items, GRNs, Analytics, Reports, Users, Settings, System Health, Audit Logs, Global Search, Procurement Dashboard, Login.

---

3. What's Broken ❌

3.1 Three API Endpoints Failing

Endpoint	Error	Root Cause
GET /audit	OperationalError	SQLite incompatibility
GET /approvals	OperationalError	SQLite incompatibility
GET /asset-incidents	OperationalError	SQLite incompatibility

These use PostgreSQL-specific SQL constructs (array_agg, complex joins) that SQLite doesn't support. **They will auto-fix once PostgreSQL is reconnected.**

3.2 Google OAuth — Why It's Failing

Real Google Sign-In does NOT work. Only the demo/mock flow works. Here's why:

Problem chain:

1. The frontend loads the Google Identity Services (GSI) SDK

2. The VITE_GOOGLE_CLIENT_ID on Render is `1015460714060-...`
3. When Google's One-Tap popup fails (wrong client ID or unauthorized origin), the code **silently falls back** to sending a mock token
4. The mock token works fine with the backend — so users get logged in, but never through real Google

To enable real Google SSO:

1. Go to Google Cloud Console → APIs → Credentials
2. Create/update OAuth 2.0 Client ID (Web app)
3. Add authorized origin: `https://expense-frontend-rxv8.onrender.com`
4. Copy new Client ID
5. Update VITE_GOOGLE_CLIENT_ID on Render frontend
6. Redeploy frontend

3.3 Database — Why SQLite Instead of PostgreSQL

Root cause: IPv6 incompatibility between Supabase and Render.

The Supabase database direct host (`db.qagextokkrxrrmowyser.supabase.co`) resolves to **only an IPv6 address** (`2406:da1a:314:...`). Render's outbound network does **not support IPv6**. This causes a `gaierror` (DNS resolution failure) on every database operation.

The fix path (connection pooler):

Supabase provides an IPv4-compatible connection pooler (`aws-0-ap-south-1.pooler.supabase.com`). However, it currently returns "tenant not found" — this likely means the pooler needs to be activated from the Supabase Dashboard → Settings → Database → Connection pooling.

Steps to reconnect PostgreSQL:

1. Go to Supabase Dashboard → Project Settings → Database
2. Enable Connection Pooling (if not enabled)
3. Copy the connection string (Transaction mode, port 6543)
4. Replace `sqlite+aiosqlite://` with the asyncpg pooler URL
5. Update DATABASE_URL on Render → Redeploy

---

4. Progress Breakdown

Area	Score	Details
Backend API	18/20	100+ endpoints. 3 failing (SQLite).
Frontend UI	16/20	25 pages. Rich dashboards & forms.
Authentication	12/20	Demo works. Real Google SSO needs config.
Deployment	14/15	Both services live. CI/CD active.
Database	5/15	Schema exists (90+ tables). Running on SQLite.
Data/Seed	7/10	6 users, basic seed data. No rich demo data.
Total	**72/100**	

---

5. Prioritized Changes

P0 — Critical

#	Change	Effort
1	Enable Supabase connection pooler from dashboard	5 min
2	Get pooler connection string + DB password	5 min
3	Update DATABASE_URL on Render to pooler URL	5 min
4	Redeploy — fixes /audit, /approvals, /asset-incidents	Auto

P1 — Important

#	Change	Effort
5	Configure real Google OAuth Client ID	30 min
6	Enable Supabase RLS on all tables	2-3 hrs
7	Seed realistic demo data	1-2 hrs
8	Set strong JWT_SECRET in production	5 min

P2 — Nice to Have

#	Change	Effort
9	Configure SMTP for email notifications	30 min
10	Set up Redis for caching	30 min

---

6. Environment Variables Audit

Backend (Render)

Variable	Value	Status
ENVIRONMENT	production	✅
DATABASE_URL	sqlite+aiosqlite:///./expense_procurement.db	⚠️ Temp
SUPABASE_URL	https://qagextokkrxrrmowyser.supabase.co	✅
SUPABASE_KEY	(anon key set)	✅
JWT_SECRET	placeholder-jwt-secret	⚠️ Insecure

Frontend (Render)

Variable	Value	Status
VITE_API_URL	https://expense-backend-1mpl.onrender.com/api/v1	✅
VITE_GOOGLE_CLIENT_ID	1015460714060-...	⚠️ Needs real ID

Variable	Value	Status
NODE_ENV	development	⚠️ Should be production

---

7. Security Warnings

1. **RLS Disabled** — Row Level Security is OFF on all 90+ Supabase tables. Anyone with the anon key can read/modify all data.
2. **JWT Secret is placeholder** — Tokens can be forged.
3. **No rate limiting** — Login endpoints have no brute-force protection.

Report generated automatically. Last verified: 9 Sep 2026, 12:00 IST.